



Assessment Cover Sheet

Assessment Title	Project _Group		
Assessment Type	Uncontrolled	Individual	Not must-pass
Due Date	15-Dec-2025	Course Code	IT7009
Course Title	Artificial Intelligence		
Internal Moderator's Name	Mahmoud Alhamad		
External Examiner's Name			

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in soft copy.
2. The deadline for this assessment is 15-Dec-2025 (11:55 PM).
3. This assessment carries 35% marks assessing CILO 1, CILO2, and CILO 3.
4. Late submission, marks will be capped at 60%. Two calendar days after the deadline, students will get 0%.
5. References consulted (if any) must be properly acknowledged and cited.

Learner ID		Date Submitted	30/12/2025
Learner Name	Abdulelah Slais		
Programme Code	IT7009		
Programme Title	Artificial Intelligence		
Lecturer's Name	Elsayed Elkenawy		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Learner Signature

Do not write beyond this line. For assessor use only.

Assessor's Name			
Marking Date		Maks Obtained	

Comments:

Executive Summary

In this practical project, we built and evaluated machine learning models to classify cyber network flows into ten categories (Benign plus nine attack types) and to surface unusual flows using unsupervised anomaly detection. Using the provided cyber traffic dataset (21,185 flows and 84 original attributes), we performed data validation, feature engineering, supervised multi-class classification (K-Nearest Neighbours and Linear Support Vector Machine), and anomaly detection with Isolation Forest. We evaluated models using accuracy and macro-averaged precision/recall/F1 to account for minority classes.

Our results show that Linear SVM achieved the best supervised performance on the held-out test set (Accuracy ≈ 0.641 , Macro-F1 ≈ 0.543), outperforming KNN both before and after tuning. For anomaly detection, Isolation Forest flagged the most unusual 5% of flows (1,060 flows), with a high concentration from the “Generic”, “DoS”, and “Exploits” classes. We interpret the supervised model using coefficient-based feature influence and permutation importance, and we interpret anomalies by comparing median shifts between anomalous and non-anomalous flows.

1. Problem and Real-World Context

Modern networks generate huge volumes of traffic, and manual inspection is not feasible at scale. Attackers exploit this reality by blending malicious activity into normal-looking traffic, using techniques such as flooding (DoS), probing (Reconnaissance), and payload delivery (Exploits, Shellcode). Automated methods that can both (i) classify known threat types and (ii) highlight suspicious outliers are valuable for security operations.

The practical goal in this project is to learn patterns from labelled network flow features and predict the traffic class for new flows. In addition, we complement supervised learning with anomaly detection to flag atypical flows that may warrant analyst triage (for example, unknown/novel behaviours, mislabelled records, or rare-but-important patterns).

2. Objectives and AI-Based Approach

We followed the project requirements by implementing both supervised and unsupervised techniques:

- Validate and prepare the dataset for modelling (data integrity checks, cleaning, and transformations).
- Engineer/organise features to improve learning and interpretability (e.g., extracting time features from Timestamp).
- Train and evaluate at least two supervised classifiers for multi-class threat classification: KNN and SVM.
- Tune model hyperparameters (e.g., K for KNN) using cross-validation and compare performance.
- Apply an unsupervised anomaly detection method (Isolation Forest) to flag the most unusual flows.

- Interpret model behaviour by identifying influential features (coefficients and permutation importance) and by analysing what distinguishes anomalies from normal traffic.

Our workflow was organised into the following phases: (A) data loading and inspection, (B) cleaning and validation, (C) feature engineering, (D) supervised modelling and tuning, (E) anomaly detection and interpretation, and (F) final evaluation and reporting. This structure aligns with the ML pipeline emphasised in the course: define the problem, prepare data, train models, evaluate results, and interpret outcomes.

3. Dataset Description

The dataset used in this project is the provided cyber network traffic CSV file. Each row represents a network flow (a sequence of packets sharing common endpoints), described by flow-level statistics such as packet lengths, inter-arrival times (IAT), flag counts, and byte/packet rates. The target variable is the categorical column “Label” with 10 classes.

Initial dataset dimensions:

- Rows: 21,185
- Columns: 84 (including identifiers and target)

Label distribution (counts): Benign (3,000), Generic (3,000), DoS (3,000), Reconnaissance (3,000), Fuzzers (3,000), Exploits (3,000), Shellcode (2,102), Backdoor (452), Analysis (385), Worms (246). While several major classes are balanced, there are minority classes with substantially fewer samples. This motivates using macro-averaged metrics during evaluation so that small classes are not overshadowed by majority classes.

... loaded dataset successfully
Shape (rows, cols): (21185, 84)

	Flow ID	Src IP	Src Port	Dst IP	Dst Port	Protocol	Timestamp	Flow Duration	Total Fwd Packet	Total Bad packets	Total Length of Fwd Packet	Total Length of Bad Packet	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Fwd Packet Length Std	Bad Packet Length Max	Bad Packet Length Min	Bad Packet Length Mean	Bad Packet Length Std	Flow Bytes/s	Flow Packets/s	Flow IAT Mean
0	175.45.176.3-149.171.126.19-60976-8022-6	175.45.176.3	60976	149.171.126.19	8022	6	22/01/2015 08:28:40 AM	234286	7	5	862.0	700.0	431.0	0.0	123.142857	210.306466	350.0	0.0	140.000000	191.702895	6667.065040	51.219450	21298.727273
1	175.45.176.2-149.171.126.18-5975-80-6	175.45.176.2	5975	149.171.126.18	80	6	18/02/2015 03:58:40 AM	5337881	63	371	368.0	473724.0	194.0	0.0	5.841270	32.518583	1460.0	0.0	1276.884097	448.250780	88816.517265	81.305672	12327.669746
2	175.45.176.2-149.171.126.15-4770-80-6	175.45.176.2	4770	149.171.126.15	80	6	18/02/2015 08:08:41 AM	557632	7	7	512.0	4476.0	256.0	0.0	73.142857	124.915209	1460.0	0.0	639.428571	659.758253	8944.967290	25.106163	42894.769231
3	175.45.176.3-149.171.126.14-6151-80-6	175.45.176.3	6151	149.171.126.14	80	6	22/01/2015 07:59:44 AM	1287210	7	7	406.0	1268.0	203.0	0.0	58.000000	99.053857	629.0	0.0	179.714286	306.920573	1292.718360	10.876236	99016.153846
4	58.166.0.1-149.171.126.0-12897-5190-6	58.166.0.1	12897	149.171.126.0	5190	6	22/01/2015 03:19:03 PM	342	1	2	0.0	0.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.000000	0.000000	0.000000	8771.929825	171.000000

`<class 'pandas.core.frame.DataFrame'>`
RangeIndex: 21185 entries, 0 to 21184
Data columns (total 84 columns):
Column Non-Null Count Dtype
0 Flow ID 21185 non-null object
1 Src IP 21185 non-null object
2 Src Port 21185 non-null int64
3 Dst IP 21185 non-null object
4 Dst Port 21185 non-null int64
5 Protocol 21185 non-null int64
6 Timestamp 21185 non-null object
7 Flow Duration 21185 non-null int64
8 Total Fwd packet 21185 non-null int64
9 Total Bad packets 21185 non-null int64
10 Total Length of Fwd Packet 21185 non-null float64
11 Total Length of Bad Packet 21185 non-null float64
12 Fwd Packet Length Max 21185 non-null float64
13 Fwd Packet Length Min 21185 non-null float64
14 Fwd Packet Length Mean 21185 non-null float64
15 Fwd Packet Length Std 21185 non-null float64
16 Bad Packet Length Max 21185 non-null float64

Figure 1: Dataset loaded successfully (head + column overview).

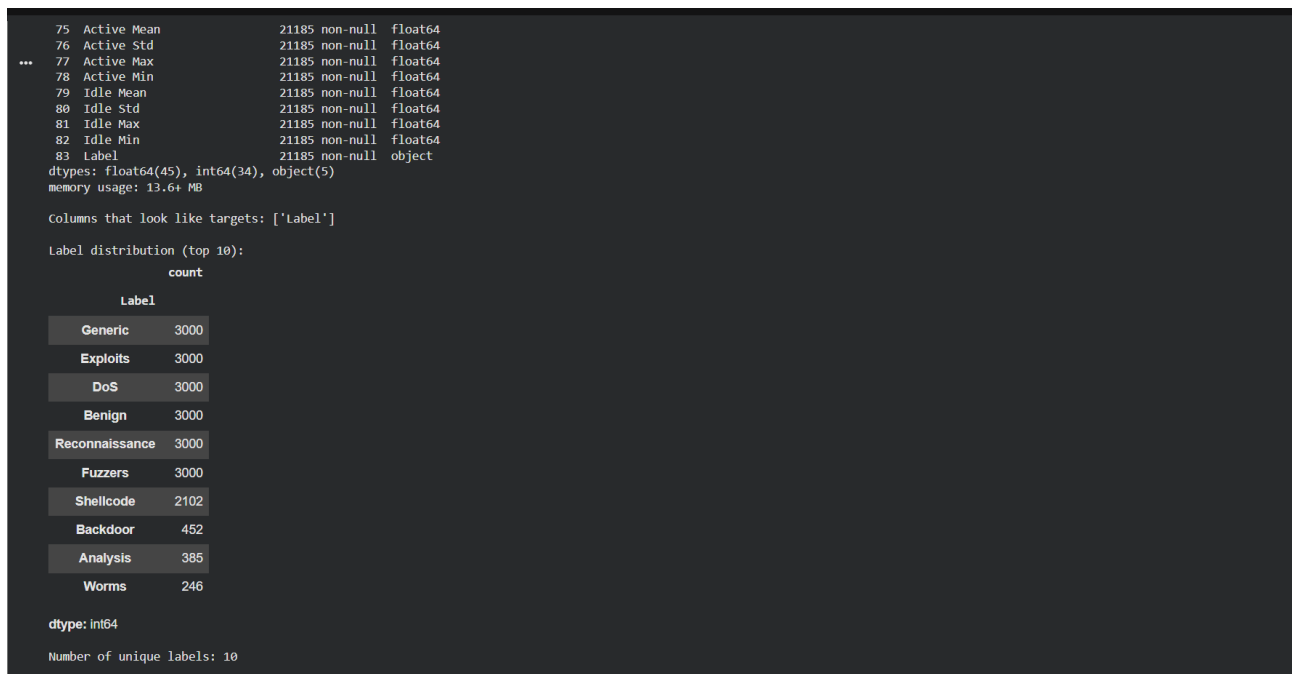


Figure 2: Target column identified (“Label”) and label distribution preview.

4. Data Preparation and Cleaning

Before modelling, we completed thorough data validation, verifying that:

- Check for missing values on a per-column basis yielded no results.
- Check for duplicate entries for each row yielded no duplicates.
- Check for infinite values showed ($\pm \infty$) could not be located in any of the numeric columns.
- Check for a mixture of data types showed that numeric features were mixed in with identifier fields (Flow ID, IP address, and Timestamp) were as expected.

It is crucial to communicate these checks even after examining the dataset because the absence or infinity of entries may create an unforeseen failure or inconsistency during model training. In addition, we ensured to convert the Timestamp field into a valid datetime type to enable consistent extraction of

time-based features.

```
... Initial shape: (21185, 84)

Missing values (non-zero only):
0

0 No missing values found ✓

dtype: object

Duplicate rows found: 0

Object columns: ['Flow ID', 'Src IP', 'Dst IP', 'Timestamp', 'Label']

Total +/-inf values across numeric columns: 0
```

Figure 3: Data integrity checks (no missing values, no duplicates, no infinite values).

```
... ✓ Cleaning complete
Rows before: 21185 | Rows after: 21185 | Removed duplicates: 0
Timestamp NaT (failed parses): 0
Total remaining missing values: 0
Total remaining +/-inf values: 0
Final shape: (21185, 84)
```

	Flow ID	Src IP	Src Port	Dst IP	Dst Port	Protocol	Timestamp	Flow Duration	Total Fwd Packet	Total Bwd packets	Total Length of Fwd Packet	Total Length of Bwd Packet	Fwd Packet Length Max	Fwd Packet Length Min
0	175.45.176.3-149.171.126.19-60976-8022-6	175.45.176.3	60976	149.171.126.19	8022	6	2015-01-22 08:28:40	234286	7	5	862.0	700.0	431.0	0.0
1	175.45.176.2-149.171.126.18-5975-80-6	175.45.176.2	5975	149.171.126.18	80	6	2015-02-18 03:58:40	5337881	63	371	368.0	473724.0	184.0	0.0
2	175.45.176.2-149.171.126.15-4770-80-6	175.45.176.2	4770	149.171.126.15	80	6	2015-02-18 08:08:41	557632	7	7	512.0	4476.0	256.0	0.0

Figure 4: Cleaning summary and preview of cleaned dataframe.

5. Feature Engineering and Selection

Feature engineering focused on improving both predictive signal and interpretability.

5.1 Timestamp-derived features

The original dataset contained a Timestamp column stored as text. Instead of discarding time information, we extracted: `ts_hour` (0–23), `ts_dayofweek` (0=Mon ... 6=Sun), and `ts_month` (1–12). After extraction, we dropped the raw Timestamp field to avoid mixing text with numeric inputs.

5.2 Dropping identifiers

We removed columns that act as identifiers rather than stable learning signals: Flow ID, Src IP, Dst IP. These identifiers can cause data leakage or memorisation and do not generalise to new traffic.

5.3 Numeric-only feature matrix and constant-feature removal

After converting and validating numeric types, we removed constant columns (features with only one unique value) because they do not contribute to classification and can add noise. In our dataset, 9 constant columns were removed, resulting in 73 numeric features used for modelling.

```
[9]
✓ Os
# =====
# Step C.1: Create features from Timestamp
# =====
# Feature Development requirement: transforming/organizing features to enhance learning/interpretability.
# We'll extract time-based features from Timestamp and keep them as numeric columns.

df_feat = df_clean.copy()

if "Timestamp" in df_feat.columns:
    df_feat["ts_hour"] = df_feat["Timestamp"].dt.hour
    df_feat["ts_dayofweek"] = df_feat["Timestamp"].dt.dayofweek # Mon=0 ... Sun=6
    df_feat["ts_month"] = df_feat["Timestamp"].dt.month
    # Drop raw timestamp after feature extraction
    df_feat = df_feat.drop(columns=["Timestamp"])

print("✅ Timestamp-derived features added (if Timestamp existed).")
print("Current shape:", df_feat.shape)
```

... ✅ Timestamp-derived features added (if Timestamp existed).
Current shape: (21185, 86)

Figure 5: Timestamp feature extraction (`ts_hour`, `ts_dayofweek`, `ts_month`) and updated shape.

[illegible]

Figure 6: Numeric-only feature matrix creation and constant-column removal (final X shape = 21,185 × 73).

6. Supervised Learning Methods

We implemented and compared two supervised multi-class classifiers.

6.1 K-Nearest Neighbours (KNN)

KNN predicts the class of a new sample by voting among the labels of its K nearest training samples in feature space. Because KNN is distance-based, feature scaling is essential; therefore, we used StandardScaler inside a Pipeline. KNN is simple and non-parametric, but can struggle in high-dimensional spaces and is sensitive to irrelevant features.

6.2 Linear Support Vector Machine (Linear SVM)

A linear SVM learns separating hyperplanes that maximise margin. For multi-class classification, a one-vs-rest strategy is used. Linear SVMs often perform strongly on high-dimensional numeric data and can be more robust than KNN when features are numerous and correlated. We used LinearSVC with scaling for stable optimisation.

6.3 Train/test split and evaluation protocol

We used an 80/20 stratified split to preserve class distribution across train and test sets (16,948 training flows; 4,237 testing flows). We report accuracy and macro-averaged precision/recall/F1. Macro-averaging treats each class equally, which is important given minority classes such as Worms, Analysis, and Backdoor.



```
... X_train: (16948, 73) X_test: (4237, 73)  
y_train distribution (top 10):
```

count	
Label	
Benign	2400
Generic	2400
DoS	2400
Reconnaissance	2400
Fuzzers	2400
Exploits	2400
Shellcode	1682
Backdoor	361
Analysis	308
Worms	197

```
dtype: int64
```

```
y_test distribution (top 10):
```

count	
Label	
Benign	600
Generic	600
Exploits	600
Fuzzers	600
Reconnaissance	600

Figure 7: Stratified train/test split and class distribution in train vs test.

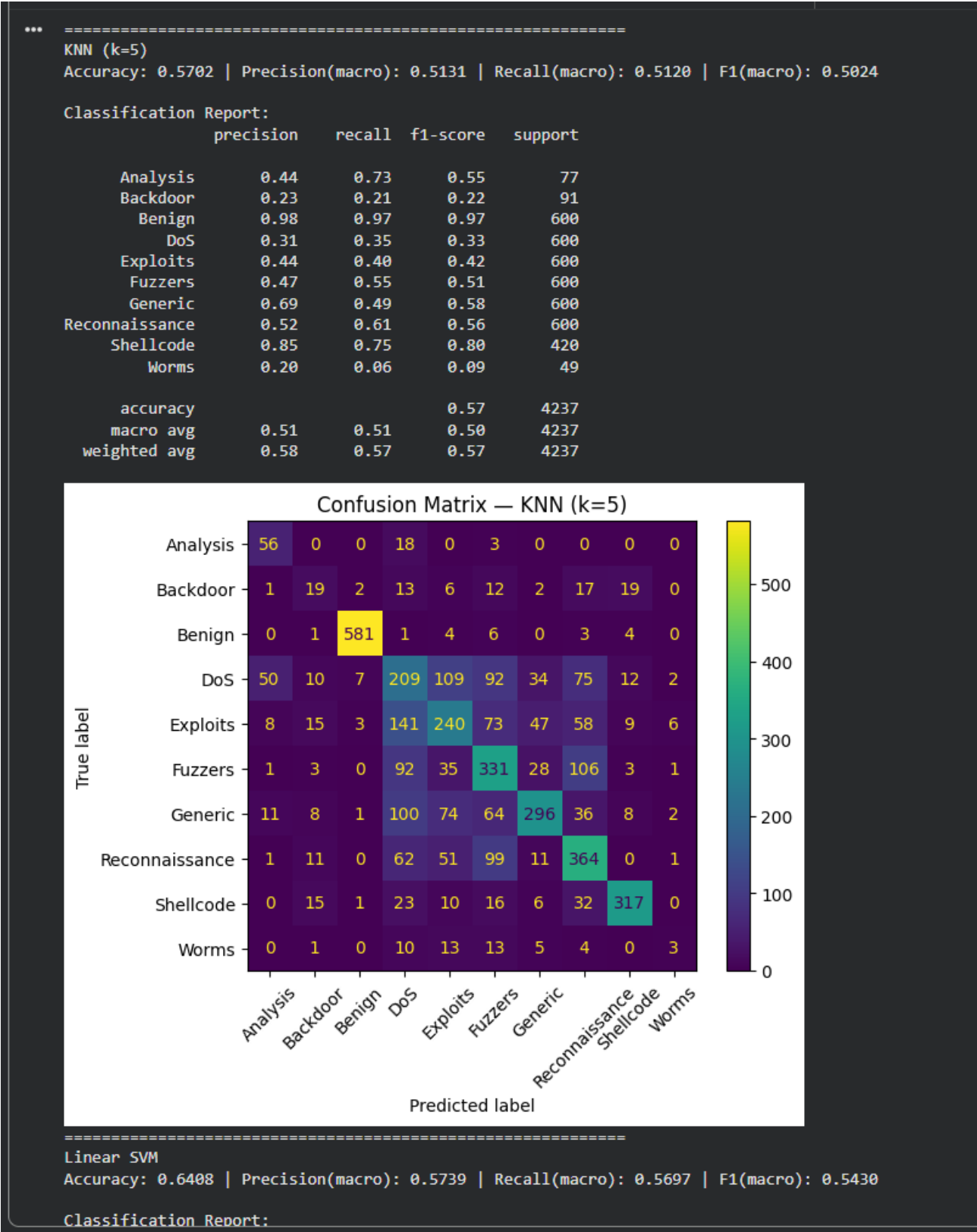


Figure 8: Baseline KNN (k=5) and Linear SVM results (classification reports + confusion matrices).

7. Hyperparameter Tuning and Model Selection

7.1 KNN tuning

We tuned K using 5-fold Stratified Cross-Validation on the training set. Candidate values were $K \in \{1, 3, 5, 7, 9, 11, 15\}$ and the optimisation metric was macro-F1 to respect minority classes. The best CV macro-F1 occurred at $K=9$ (≈ 0.494).

7.2 Final supervised comparison

After tuning, KNN improved slightly on test accuracy ($\approx 0.570 \rightarrow \approx 0.573$), but macro-F1 remained around ≈ 0.504 . Linear SVM remained the strongest overall model with ≈ 0.641 accuracy and ≈ 0.543 macro-F1. The improvement of SVM suggests that margin-based separation is more effective than local neighbourhood voting for this feature space.

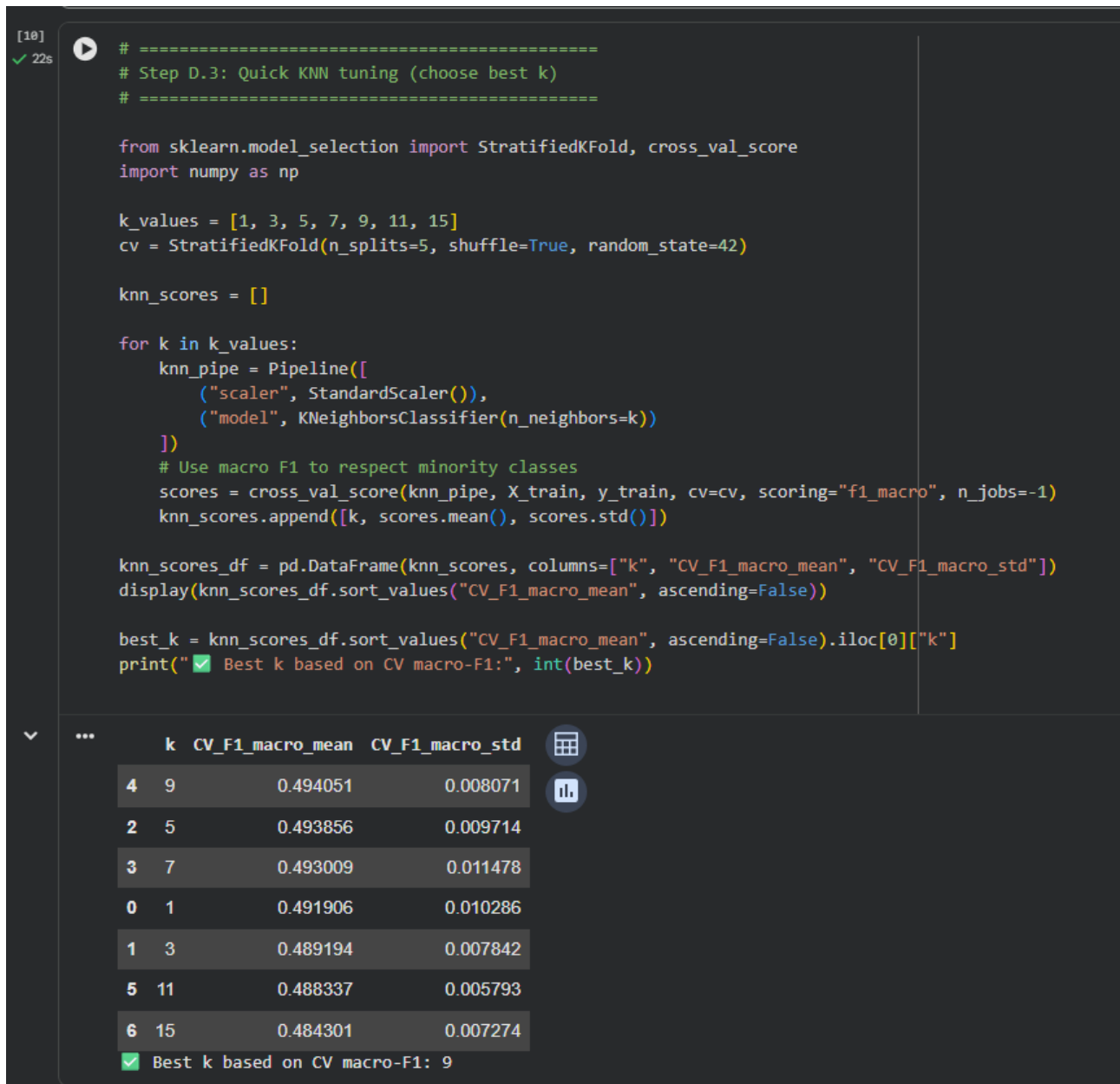


Figure 9: KNN cross-validation tuning results (best K=9 by macro-F1).

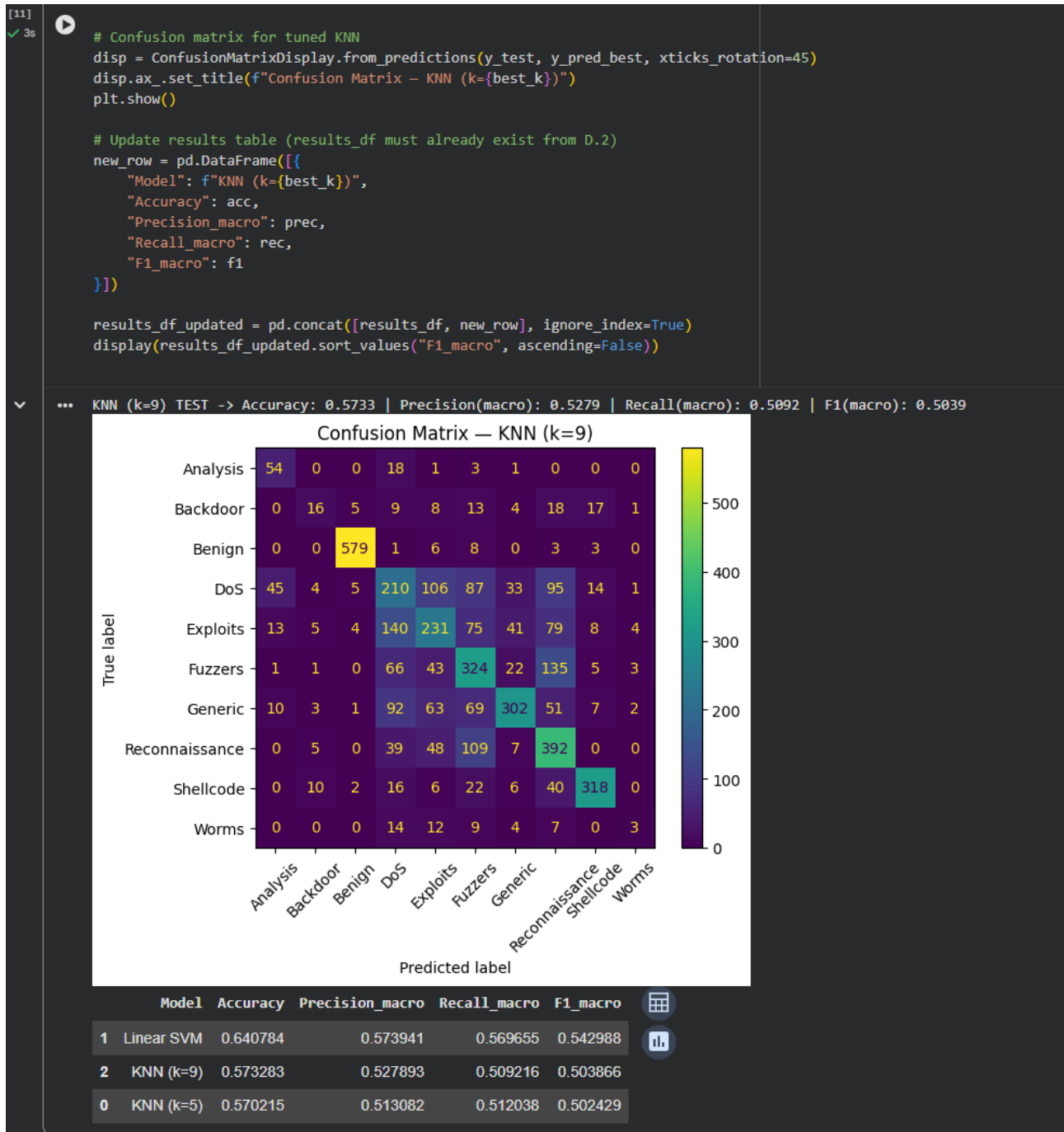


Figure 10: Tuned KNN (k=9) evaluation and updated model comparison table.

```

[20]
✓ 4s
import pandas as pd
from sklearn.metrics import classification_report, accuracy_score, precision_score, recall_score, f1_score

def eval_model(name, model, X_test, y_test):
    y_pred = model.predict(X_test)
    row = {
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision_macro": precision_score(y_test, y_pred, average="macro", zero_division=0),
        "Recall_macro": recall_score(y_test, y_pred, average="macro", zero_division=0),
        "F1_macro": f1_score(y_test, y_pred, average="macro", zero_division=0),
    }
    report = classification_report(y_test, y_pred, output_dict=True, zero_division=0)
    per_class = (pd.DataFrame(report).T
                 .drop(index=["accuracy", "macro avg", "weighted avg"], errors="ignore")
                 .sort_values("f1-score"))
    return row, per_class

# expects these exist from earlier steps:
# svm_pipe (final Linear SVM pipeline), knn_best (tuned KNN pipeline), X_test, y_test

rows = []
row_svm, perclass_svm = eval_model("Linear SVM", svm_pipe, X_test, y_test)
row_knn, perclass_knn = eval_model("KNN (tuned)", knn_best, X_test, y_test)

rows.extend([row_svm, row_knn])
supervised_summary = pd.DataFrame(rows).sort_values("F1_macro", ascending=False)

print("Supervised model comparison (higher is better):")
display(supervised_summary)

print("\nWorst-performing classes (by F1) – Linear SVM:")
display(perclass_svm.head(5))

# Save for report
supervised_summary.to_csv("supervised_model_comparison.csv", index=False)
perclass_svm.to_csv("svm_per_class_report.csv")
perclass_knn.to_csv("knn_per_class_report.csv")
print("✓ Saved: supervised_model_comparison.csv, svm_per_class_report.csv, knn_per_class_report.csv")

```

... Supervised model comparison (higher is better):

	Model	Accuracy	Precision_macro	Recall_macro	F1_macro
0	Linear SVM	0.640784	0.573941	0.569655	0.542988
1	KNN (tuned)	0.573283	0.527893	0.509216	0.503866

Worst-performing classes (by F1) – Linear SVM:

	precision	recall	f1-score	support
Worms	0.000000	0.000000	0.000000	49.0
Backdoor	0.680000	0.186813	0.293103	91.0
DoS	0.471125	0.258333	0.333692	600.0
Exploits	0.600950	0.421667	0.495593	600.0
Fuzzers	0.479282	0.578333	0.524169	600.0

✓ Saved: supervised_model_comparison.csv, svm_per_class_report.csv, knn_per_class_report.csv

Figure 11: Supervised model comparison and worst-performing classes (by F1).

7.3 Summary table (test set)

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
KNN (k=5)	0.5702	0.5131	0.5120	0.5024
KNN (k=9 tuned)	0.5733	0.5279	0.5092	0.5039
Linear SVM	0.6408	0.5739	0.5697	0.5430

Interpretation: Linear SVM provides the best balance across classes according to macro metrics. KNN performs strongly for Benign traffic but struggles to separate multiple attack categories that overlap in flow statistics, especially when minority classes are present.

8. Unsupervised Anomaly Detection (Isolation Forest)

Supervised classifiers rely on known labels. To complement them, we applied Isolation Forest (IF) to the numeric feature space to flag unusual flows. Isolation Forest isolates anomalies by randomly partitioning the feature space; points that require fewer splits to isolate are considered more anomalous.

8.1 Contamination (expected anomaly rate)

Because we do not have ground-truth anomaly labels, we set contamination = 0.05 (flag the most unusual 5% of flows). This provides a practical threshold that is easy to interpret and aligns with triage-style anomaly detection where analysts review the top-ranked outliers.

8.2 Outputs

Isolation Forest marked 1,060 flows as anomalous ($\approx 5.00\%$ of the dataset). We analysed the distribution of known labels among flagged anomalies and ranked flows by anomaly score to identify the most extreme outliers.

```

[12]
✓ 7s
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd

# -----
# Step E.1: Isolation Forest
# -----
# Criteria choice (required by brief): we flag the most unusual 5% of flows as anomalies.
# This is a practical threshold when you don't have ground-truth anomaly labels.

ANOMALY_RATE = 0.05 # 5% (adjustable; we'll justify in report as a decision threshold)

iso_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("iso", IsolationForest(
        n_estimators=300,
        contamination=ANOMALY_RATE,
        random_state=42,
        n_jobs=-1
    ))
])

iso_pipe.fit(X_anom)

# IsolationForest outputs: -1 anomaly, +1 normal
pred_flag = iso_pipe.named_steps["iso"].predict(iso_pipe.named_steps["scaler"].transform(X_anom))

# Decision function: higher = more normal; lower = more anomalous
scores = iso_pipe.named_steps["iso"].decision_function(iso_pipe.named_steps["scaler"].transform(X_anom))

anomaly_mask = (pred_flag == -1)
anomaly_idx = X_anom.index[anomaly_mask]

print("Total samples:", len(X_anom))
print("Anomalies flagged:", anomaly_mask.sum(), f"({anomaly_mask.mean()*100:.2f}%)")

# Store for later steps
anomaly_scores = pd.Series(scores, index=X_anom.index, name="anomaly_score")

... Total samples: 21185
Anomalies flagged: 1060 (5.00%)

```

Figure 12: Isolation Forest configuration and anomaly rate (5% flagged).

...

Label distribution among flagged anomalies (%):

count

Label

Generic	42.55
DoS	16.98
Exploits	16.60
Fuzzers	10.66
Benign	9.91
Worms	1.98
Reconnaissance	0.38
Shellcode	0.38
Backdoor	0.28
Analysis	0.28

dtype: float64

	Flow ID	Src IP	Dst IP	Src Port	Dst Port	Protocol	Label	anomaly_score
15408	175.45.176.2-149.171.126.15-4664-25-6	175.45.176.2	149.171.126.15	4664	25	6	DoS	-0.214731
12832	175.45.176.3-149.171.126.11-38156-25-6	175.45.176.3	149.171.126.11	38156	25	6	DoS	-0.213540
12427	175.45.176.0-149.171.126.10-28085-25-6	175.45.176.0	149.171.126.10	28085	25	6	DoS	-0.213218
6548	175.45.176.3-149.171.126.12-26969-25-6	175.45.176.3	149.171.126.12	26969	25	6	DoS	-0.212897
19267	175.45.176.2-149.171.126.16-39366-25-6	175.45.176.2	149.171.126.16	39366	25	6	DoS	-0.212527
19749	175.45.176.1-149.171.126.13-47895-80-6	175.45.176.1	149.171.126.13	47895	80	6	Fuzzers	-0.211803
17489	175.45.176.3-149.171.126.14-11360-25-6	175.45.176.3	149.171.126.14	11360	25	6	DoS	-0.211337
2276	175.45.176.1-149.171.126.15-2002-25-6	175.45.176.1	149.171.126.15	2002	25	6	DoS	-0.209602
697	59.166.0.7-149.171.126.9-55783-6881-6	59.166.0.7	149.171.126.9	55783	6881	6	Benign	-0.207643
20853	175.45.176.2-149.171.126.10-30505-25-6	175.45.176.2	149.171.126.10	30505	25	6	DoS	-0.207245

Figure 13: Label distribution among flagged anomalies and example anomalous flows with scores.

[15]

✓ Os



```
# -----  
# Step E.3 (Correct): Feature shifts (anomalies vs normal)  
# -----  
# Compare anomalies to NON-anomalies using robust medians, then rank features  
# by the absolute median shift. This helps explain "what makes them weird".  
  
import numpy as np  
import pandas as pd  
  
# X_anom: numeric feature matrix used for anomaly detection  
# anomaly_mask: boolean array/Series where True = anomaly (pred_flag == -1)  
  
# Ensure anomaly_mask is aligned to X_anom index (safe if you created it from X_anom)  
anomaly_mask = pd.Series(anomaly_mask, index=X_anom.index)  
  
anomaly_idx = X_anom.index[anomaly_mask]  
normal_idx = X_anom.index[~anomaly_mask] # ✓ normal = NOT anomaly  
  
anom_median = X_anom.loc[anomaly_idx].median(numeric_only=True)  
norm_median = X_anom.loc[normal_idx].median(numeric_only=True)  
  
median_shift = (anom_median - norm_median).abs().sort_values(ascending=False)  
  
print("Top 15 features with biggest median shift (anomalies vs normal):")  
display(median_shift.head(15))
```



... Top 15 features with biggest median shift (anomalies vs normal):

0

Flow Duration	8.158518e+06
Fwd IAT Total	8.066566e+06
Bwd IAT Total	2.879622e+06
Fwd IAT Max	1.466111e+06
Flow IAT Max	1.336868e+06
Packet Length Variance	3.058570e+05
Bwd IAT Max	2.490195e+05
Fwd IAT Std	1.171315e+05
Flow IAT Std	6.977312e+04
Fwd IAT Mean	6.134040e+04
Flow Bytes/s	2.820236e+04
Bwd Bulk Rate Avg	2.402150e+04
Bwd Bytes/Bulk Avg	2.282800e+04
Flow IAT Mean	5.323745e+03
Bwd IAT Std	4.889314e+03

dtype: float64

Figure 14: Feature median shifts (anomalies vs non-anomalies): top features by absolute shift.

Key observation: A large share of flagged anomalies belong to the “Generic” class (~42.55%), followed by DoS and Exploits. However, a non-trivial portion of anomalies are labelled “Benign” (~9.91%), which is expected: benign traffic can contain rare events that are statistically unusual but not malicious. In real operations, anomaly detection is best used as a triage tool rather than a definitive classifier.

9. Interpretation and Feature Analysis

Understanding why a model predicts a given class is important in cybersecurity. We used two complementary interpretability methods for the best supervised model (Linear SVM).

9.1 Coefficient-based global importance

Linear SVM provides per-class linear weights for each feature. We computed a global importance score by taking the mean absolute coefficient across classes. This highlights which inputs generally influence the decision boundaries.

9.2 Permutation importance

Permutation importance measures how much performance drops when we randomly shuffle a feature, breaking its relationship to the label. We computed macro-F1 permutation importance on a test subset for computational efficiency. This provides a model-agnostic check and helps validate whether coefficient ranking aligns with predictive usefulness.

9.3 Per-class drivers

We extracted top positive and negative coefficients per class to identify features that push predictions toward or away from each threat type.

[17]
✓ Os



```
# =====  
# Step F.2 – Coefficient-based feature influence (interpretable)  
# =====  
  
import numpy as np  
import pandas as pd  
  
scaler = svm_pipe.named_steps["scaler"]  
svm = svm_pipe.named_steps["model"]  
  
feature_names = X.columns.tolist()  
  
# coef_ is in "scaled-feature space". Convert to original feature units:  
# If  $z = (x - \text{mean})/\text{scale}$ , then decision uses  $w_z$ ; in original units  $w_x = w_z / \text{scale}$ .  
W_scaled = svm.coef_ # shape: (n_classes, n_features)  
W = W_scaled / scaler.scale_ # convert weights to original units  
  
# Global importance: mean abs weight across one-vs-rest classifiers  
global_importance = np.mean(np.abs(W), axis=0)  
  
global_imp_df = pd.DataFrame({  
    "feature": feature_names,  
    "importance_mean_abs_coef": global_importance  
}).sort_values("importance_mean_abs_coef", ascending=False)  
  
print("Top 20 global features (mean |coef| across classes):")  
display(global_imp_df.head(20))  
  
# Save for report  
global_imp_df.to_csv("svm_global_feature_importance.csv", index=False)  
print("✅ Saved: svm_global_feature_importance.csv")
```



... Top 20 global features (mean |coef| across classes):

	feature	importance_mean_abs_coef	
32	Fwd PSH Flags	1.877361	
44	RST Flag Count	1.687527	
72	ts_month	0.174183	
42	FIN Flag Count	0.150461	
47	Down/Up Ratio	0.132899	
71	ts_dayofweek	0.128230	
2	Protocol	0.105399	
61	Fwd Seg Size Min	0.071657	
45	PSH Flag Count	0.061301	
43	SYN Flag Count	0.052803	
13	Bwd Packet Length Min	0.017207	
21	Flow IAT Min	0.009821	
31	Bwd IAT Min	0.007040	
70	ts_hour	0.005944	
15	Bwd Packet Length Std	0.005424	
26	Fwd IAT Min	0.003664	
9	Fwd Packet Length Min	0.003577	
37	Packet Length Min	0.003463	

Figure 15: Coefficient-based global feature importance (mean $|coef|$ across classes).

[19]
✓ 24s



```
# =====  
# Step F.4 – Permutation importance (model-agnostic validation)  
# =====  
  
from sklearn.inspection import permutation_importance  
  
# Use a subset if you want it faster (keeps results stable enough for reporting)  
X_pi = X_test  
y_pi = y_test  
if len(X_pi) > 2000:  
    idx = X_pi.sample(2000, random_state=42).index  
    X_pi = X_pi.loc[idx]  
    y_pi = y_pi.loc[idx]  
  
pi = permutation_importance(  
    svm_pipe, X_pi, y_pi,  
    scoring="f1_macro",  
    n_repeats=5,  
    random_state=42,  
    n_jobs=-1  
)  
  
perm_df = pd.DataFrame({  
    "feature": feature_names,  
    "perm_importance_mean": pi.importances_mean,  
    "perm_importance_std": pi.importances_std  
}).sort_values("perm_importance_mean", ascending=False)  
  
print("Top 20 permutation-importance features (by macro-F1 drop):")  
display(perm_df.head(20))  
  
perm_df.to_csv("svm_permutation_importance.csv", index=False)  
print("✅ Saved: svm_permutation_importance.csv")
```



... Top 20 permutation-importance features (by macro-F1 drop):

	feature	perm_importance_mean	perm_importance_std
12	Bwd Packet Length Max	0.170887	0.008734
61	Fwd Seg Size Min	0.149589	0.008606
15	Bwd Packet Length Std	0.148454	0.009845
38	Packet Length Max	0.141906	0.006637
41	Packet Length Variance	0.135482	0.003601
40	Packet Length Std	0.131383	0.008762
39	Packet Length Mean	0.130689	0.006431
1	Dst Port	0.128795	0.005056
48	Average Packet Size	0.125507	0.008026
2	Protocol	0.108356	0.004192
11	Fwd Packet Length Std	0.098934	0.009447
8	Fwd Packet Length Max	0.094952	0.003789



Figure 16: Permutation importance (macro-F1 drop) for top features.

[18]
✓ Os



```
# Step F.3 – Per-class top features (what drives each threat type)
# =====

classes = svm.classes_
per_class_tables = {}

for i, cls in enumerate(classes):
    w = W[i]
    s = pd.Series(w, index=feature_names)

    top_pos = s.sort_values(ascending=False).head(10) # pushes prediction toward this class
    top_neg = s.sort_values(ascending=True).head(10)  # pushes prediction away from this class

    per_class_tables[cls] = (top_pos, top_neg)

print("✅ Per-class feature drivers computed.")
print("Example output for one class (first class in svm.classes_):")
example_cls = classes[0]
print("Class:", example_cls)
print("\nTop + features:")
display(per_class_tables[example_cls][0].to_frame("coef"))
print("\nTop - features:")
display(per_class_tables[example_cls][1].to_frame("coef"))
```



✅ Per-class feature drivers computed.
Example output for one class (first class in svm.classes_):
Class: Analysis

Top + features:

	coef	
Fwd PSH Flags	2.632591	
RST Flag Count	1.345152	
ts_month	0.649333	
ts_dayofweek	0.554019	
SYN Flag Count	0.192646	
FIN Flag Count	0.158016	
Protocol	0.052493	
ts_hour	0.021166	
Bwd IAT Min	0.017866	
Down/Up Ratio	0.016157	

Top - features:

	coef	
Flow IAT Min	-0.029478	
Fwd Seg Size Min	-0.008885	
Fwd Segment Size Avg	-0.006604	
Fwd Packet Length Mean	-0.006604	
Subflow Bwd Packets	-0.002381	
Fwd Act Data Pkts	-0.002132	
Bwd Packet/Bulk Avg	-0.001822	
Subflow Fwd Packets	-0.001617	
Bwd Packet Length Max	-0.001572	

Figure 17: Example per-class feature drivers (positive and negative coefficients for one class).

Interpretation summary:

- TCP flag count features (e.g., PSH, RST, FIN, SYN) appear as strong drivers, which is plausible because different attack behaviours have characteristic flag patterns.
- Packet length statistics and segment sizes are consistently important, reflecting traffic volume and burst patterns.
- Destination port and protocol are also influential (permutation ranking), consistent with attacks targeting specific services.
- Time-derived features (ts_month, ts_dayofweek) appear in coefficient rankings, suggesting temporal structure in the capture period; these may not generalise across different capture windows and should be treated cautiously.

10. Discussion: Implications, Challenges, Limitations, Improvements

10.1 Practical implications

Our best supervised model (Linear SVM) can be used as a first-pass classifier to label incoming flows as Benign or as specific attack categories, enabling automated alert routing and prioritisation. In parallel, Isolation Forest can rank unusual flows for review, which is useful for identifying novel patterns not covered by known labels.

10.2 Challenges encountered

- High dimensionality (73 numeric features) increases the chance of overlapping classes and noisy distances for KNN.
- Minority classes (Worms, Analysis, Backdoor) reduce macro-F1 and are difficult to learn reliably. In our results, Worms had near-zero F1 in Linear SVM, indicating almost no correct predictions for that class.
- Several attack types show similar flow-level signatures, leading to confusion in the confusion matrices.

10.3 Limitations

- Evaluation uses a single hold-out split; repeated splits or nested CV would provide stronger confidence.
- Temporal features may encode capture-specific artefacts that limit generalisation.
- Isolation Forest anomalies are statistical outliers and may include benign rare events.

10.4 Possible improvements

- Hyperparameter optimisation for SVM (C tuning) and considering kernels if feasible.
- Class-weighting or resampling to improve minority-class recall.
- Feature selection/dimensionality reduction to reduce noise for distance-based methods.
- Probability calibration and thresholding for operational deployment.
- Local explanations (e.g., SHAP) for case-by-case interpretability if allowed.

11. Overall Findings and Final Reflections

Our experiments show that a linear margin-based classifier (Linear SVM) provides better multi-class classification performance than KNN on this cyber flow dataset, especially when judged by macro-averaged metrics that treat all classes fairly. KNN benefited from tuning but improvements were modest, suggesting that local neighbourhood structure is not sufficiently clean in the chosen feature representation.

The unsupervised anomaly analysis adds a second lens: anomaly detection highlights statistically unusual flows, which is useful where labels are incomplete or adversaries evolve tactics. The most anomalous flows were associated with strong shifts in duration and inter-arrival-time features, indicating atypical timing/burst patterns.

This project reinforced the end-to-end ML workflow: validate data, engineer features, build baselines, tune, evaluate with appropriate metrics, and interpret the outcome in the domain context.

References

1. IT7009 – Practical Project Brief 1 (unit-provided assessment document).
2. IT7009 lecture materials (Weeks 1–11), focusing on supervised classification, evaluation metrics, and data preparation.

Appendix A: Reproducibility and Saved Artefacts

During execution, we saved intermediate artefacts for reporting and verification, including: `svm_global_feature_importance.csv`, `svm_permutation_importance.csv`, `supervised_model_comparison.csv`, `svm_per_class_report.csv`, `knn_per_class_report.csv`, `anomaly_label_distribution_pct.csv`, `anomaly_rate_by_label.csv`, and `top200_anomalies.csv`. We also ran a final QA checklist to confirm shapes, absence of missing values, and presence of expected outputs.

23]
Os

```
import os
import numpy as np
import pandas as pd

print("=== FINAL QA CHECKS ===")

# 1) Core shapes
print("df_clean:", df_clean.shape)
print("X:", X.shape, "| y:", y.shape)
print("X_train:", X_train.shape, "| X_test:", X_test.shape)

# 2) Data integrity
print("\nMissing values in X:", int(X.isna().sum().sum()))
print("Non-numeric columns in X:", X.select_dtypes(exclude=[np.number]).columns.tolist())
print("Duplicate rows in df_clean:", int(df_clean.duplicated().sum()))

# 3) Labels
print("\n#Classes in y:", y.nunique())
print("Class counts (train):")
display(y_train.value_counts())
print("Class counts (test):")
display(y_test.value_counts())

# 4) Model results must exist
print("\nSupervised summary exists:", "supervised_summary" in globals())
if "supervised_summary" in globals():
    display(supervised_summary)

# 5) Unsupervised results must exist
print("\nAnomaly results exist:", all(k in globals() for k in ["anomaly_mask", "anomaly_scores"]))
if "anomaly_mask" in globals():
    print("Anomaly rate:", float(pd.Series(anomaly_mask).mean()))

# 6) Check saved files (created during steps F & G)
expected_files = [
    "svm_global_feature_importance.csv",
    "svm_permutation_importance.csv",
    "supervised_model_comparison.csv",
    "svm_per_class_report.csv",
    "knn_per_class_report.csv",
    "anomaly_label_distribution_pct.csv",
    "anomaly_rate_by_label.csv",
    "top200_anomalies.csv"
]
present = {f: os.path.exists(f) for f in expected_files}
print("\nSaved files status:")
display(pd.DataFrame({"file": list(present.keys()), "exists": list(present.values())}))
```

```
... === FINAL QA CHECKS ===
df_clean: (21185, 84)
X: (21185, 73) | y: (21185,)
X_train: (16948, 73) | X_test: (4237, 73)
```

```
Missing values in X: 0
Non-numeric columns in X: []
Duplicate rows in df_clean: 0
```

```
#Classes in y: 10
Class counts (train):
```

	count
Label	
Benign	2400
Generic	2400
DoS	2400
Reconnaissance	2400
Fuzzers	2400

Figure A1: Final QA checks confirming dataset integrity and saved artefacts.



```
anom_rate = float(pd.Series(anomaly_mask, index=X_anom.index).mean())
print(f"Flagged anomaly rate: {anom_rate*100:.2f}%")

print("\nTop 10 global SVM features (coef-based):")
try:
    global_imp_df = pd.read_csv("svm_global_feature_importance.csv")
    display(global_imp_df.head(10))
except Exception as e:
    print("Could not load svm_global_feature_importance.csv:", e)

print("\nTop 10 permutation-importance features:")
try:
    perm_df = pd.read_csv("svm_permutation_importance.csv")
    display(perm_df.head(10))
except Exception as e:
    print("Could not load svm_permutation_importance.csv:", e)
```

*** === FINAL PRACTICAL SUMMARY ===

Best supervised model (from your results):

Model Linear SVM

Accuracy 0.640784

Precision_macro 0.573941

Recall_macro 0.569655

F1_macro 0.542988

Name: 0, dtype: object

Anomaly detection:

Flagged anomaly rate: 5.00%

Top 10 global SVM features (coef-based):

	feature	importance_mean_abs_coef
0	Fwd PSH Flags	1.877361
1	RST Flag Count	1.687527
2	ts_month	0.174183
3	FIN Flag Count	0.150461
4	Down/Up Ratio	0.132899
5	ts_dayofweek	0.128230
6	Protocol	0.105399
7	Fwd Seg Size Min	0.071657
8	PSH Flag Count	0.061301



Figure A2: Final practical summary (best model, anomaly rate, top features).